SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: Total Marks: 30

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
   <input type="text" name="name">
   <input type="email" name="email">
   <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

1. Identify appropriate HTML5 semantic elements missing in the structure.
2. Modify the structure to make it standards-compliant.
3. Interpret the HTTP request generated during form submission.
4. Analyze the HTTP response cycle from server to browser.
5. Suggest improvements for usability and accessibility.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

1. Identify improper nesting and missing closing tags.
2. Analyze how different browsers may interpret this structure.
3. Identify missing semantic elements (e.g., header, section).
4. Provide a corrected W3C-compliant HTML structure.
5. Suggest techniques to ensure cross-browser compatibility.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

1. Identify issues in the form configuration.
2. Analyze why the HTTP request is not triggered.
3. Interpret the role of GET method in this scenario.
4. Propose a corrected implementation.
5. Suggest improvements for secure data transmission.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

ANSWERS

Answer 1: Online Symposium Portal

1. Missing HTML5 semantic elements. The snippet contains only a bare form, so the page has no meaningful structure. The following are missing:

- **<!DOCTYPE html>, <html lang>, <head> with <meta charset>, <meta viewport> and <title>**: the document skeleton required for standards mode, correct character encoding and mobile rendering.
- **<header>**: holds the portal title and branding.
- **<main>**: marks the primary content of the page (one per page).
- **<section>** with a heading: groups the registration area as a thematic block.
- **<fieldset> and <legend>**: group related form controls and describe them.
- **<label>**: associates visible text with every input (no labels exist at all).
- **<footer>**: holds copyright and contact details.

2. Standards-compliant structure.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Symposium Registration</title>
</head>
<body>
  <header>
    <h1>Online Symposium Registration</h1>
  </header>
  <main>
    <section aria-labelledby="reg-title">
      <h2 id="reg-title">Register Now</h2>
      <form action="/register" method="POST">
        <fieldset>
          <legend>Participant Details</legend>
          <label for="name">Full Name</label>
          <input type="text" id="name" name="name"
                 required autocomplete="name">
          <label for="email">Email Address</label>
          <input type="email" id="email" name="email"
                 required autocomplete="email">
          <button type="submit">Submit</button>
        </fieldset>
      </form>
    </section>
  </main>
  <footer>
    <p>&copy; 2026 Symposium Organizing Committee</p>
  </footer>
</body>
</html>
```

3. HTTP request generated on submission. The form uses method POST, so the browser sends the data in the request body instead of the URL. The button has no type, so it defaults to *submit* inside a form. Both inputs have a *name* attribute, so both values are included. The request looks like this:

```
POST /register HTTP/1.1
Host: symposium.college.edu
Content-Type: application/x-www-form-urlencoded
Content-Length: 41
Cookie: session=abc123

name=Ravi+Kumar&email=ravi%40example.com
```

The request line gives the method, path and protocol version. The headers describe the host, the encoding of the body and its size, and the blank line separates headers from the body. The body holds name=value pairs joined by & and URL-encoded (space becomes +, @ becomes %40).

4. HTTP response cycle from server to browser.

1. The server receives the request, routes it by path (/register) and method (POST) to a handler.
2. The handler parses the body, validates the fields, sanitizes them and stores the registration in a database.
3. The server builds a response with a status line, headers and optionally a body. On success it commonly sends **302 Found** or **303 See Other** with *Location: /thank-you* (the Post/Redirect/Get pattern). This prevents a duplicate registration if the user refreshes the page. A server may instead send **200 OK** with the confirmation HTML directly.
4. The browser follows the redirect with a new GET request to /thank-you and receives **200 OK** with *Content-Type: text/html*.
5. The browser parses the HTML into the DOM, fetches linked CSS, JavaScript and images, builds the render tree and paints the confirmation page.
6. On failure, the server returns **400/422** (invalid input) or **500** (server error), and the page should show a clear message.

5. Improvements for usability and accessibility.

- Give every input a visible <*label*> linked by *for* and *id*. Placeholders alone are not a substitute because they disappear while typing and are poorly supported by screen readers.
- Use *required*, correct input types (email, tel) and *autocomplete* hints for faster, error-free entry.
- Validate on both client and server. Client checks give instant feedback, but only server checks are trustworthy.
- Show clear, specific error messages next to the field and announce them with *aria-live* for screen readers.
- Keep the whole form usable by keyboard (logical tab order, visible focus outline) and ensure sufficient colour contrast.
- Make the layout responsive and use touch-friendly button sizes. Name the button by its action (for example "Register for Symposium").
- Serve the page over HTTPS and add a CSRF token to protect the POST request.

Answer 2: Cross-Browser Compatibility Issue

1. Improper nesting and missing closing tags.

- **<h1>** is never closed. A heading must end with </h1> before any block content follows.
- **<p>** is never closed explicitly. HTML5 allows the closing tag to be omitted in some cases, but relying on this is poor practice and invalid in XHTML.
- **The <!DOCTYPE html> declaration is missing**, and there is no lang attribute, charset or viewport meta tag.
- The unclosed h1 and p are only terminated indirectly when the parser meets , so the nesting is accidental rather than intended.

2. How browsers may interpret the structure. Without a doctype, every browser switches to *quirks mode*, in which the box model, table sizing, line height and font inheritance follow legacy rules that differ slightly between engines. This is the main source of the inconsistent layout. In addition, an HTML5 parser does not close an open <h1> when it sees a <p> start tag, so the paragraph *About us* becomes a child of the heading and inherits the large bold heading style. Both elements are closed only when is reached. Older or non-conforming parsers may close the heading earlier and render the paragraph normally. The DOM therefore differs by browser version, and so does the layout. An XML-based (XHTML) parser would refuse to render the page at all because it is not well formed.

3. Missing semantic elements. The page uses a generic for everything. It should use <header> for the title area, <nav> for site navigation, <main> for the primary content, <section> or <article> for the About Us block, and <footer> for closing information. These elements give meaning to browsers, search engines and assistive technology.

4. Corrected W3C-compliant HTML structure.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Company</title>
</head>
<body>
  <header>
    <h1>Welcome</h1>
  </header>
  <main>
    <section>
      <h2>About Us</h2>
      <p>About us</p>
    </section>
  </main>
  <footer>
    <p>&copy; 2026 Company</p>
  </footer>
</body>
</html>
```

5. Techniques for cross-browser compatibility.

- Always declare *<!DOCTYPE html>* so that all browsers use standards mode.
- Validate the markup and CSS with the W3C validators and fix every error and warning.
- Use a CSS reset or normalize.css to remove default style differences between browsers.
- Apply *box-sizing: border-box* globally and use Flexbox or Grid with vendor prefixes added by Autoprefixer.
- Use feature detection (*@supports*, Modernizr) and progressive enhancement rather than browser sniffing.
- Provide fallbacks for newer features and check support on caniuse.com.
- Test on Chrome, Firefox, Safari and Edge, on real mobile devices and with tools such as BrowserStack, and use responsive design with relative units.

Answer 3: Form Submission Failure

1. Issues in the form configuration.

1. **button type="button"**: a plain button has no default action, so it does not submit the form.

2. **method="GET"** for a login form: credentials would be appended to the URL (/submit?username=..&password=..) and exposed in the address bar, browser history, server logs and the Referer header.
3. No <*label*> elements, no *id* attributes, no *required* and no *autocomplete* hints, which hurts accessibility and usability.
4. No HTTPS, CSRF protection or server-side validation is indicated.

2. Why the HTTP request is not triggered. Form submission happens only when the user activates a control of type *submit* (a <button> without a type, <button type="submit"> or <input type="submit">) or when JavaScript calls form.submit(). A <button type="button"> is a generic button meant to run JavaScript through an event handler. No script is attached here, so clicking Login does nothing and the browser never builds or sends an HTTP request.

3. Role of the GET method in this scenario. GET is designed to retrieve a resource. It is safe and idempotent, and it carries data in the query string. If the button were fixed but GET kept, the browser would send *GET /submit?username=admin&password=1234 HTTP/1.1*. The password would then be visible on screen, stored in history, bookmarked, cached and written to logs, and URLs have length limits. GET is therefore appropriate for searching or filtering, but unsuitable for login or any request that changes state or carries secrets.

4. Corrected implementation.

```
<form action="/submit" method="POST">
   <label for="username">Username</label>
   <input type="text" id="username" name="username"
        required autocomplete="username">
   <label for="password">Password</label>
   <input type="password" id="password" name="password"
        required autocomplete="current-password">
   <button type="submit">Login</button>
</form>
```

5. Improvements for secure data transmission.

- Serve the whole site over **HTTPS (TLS)** and enable HSTS so credentials are encrypted in transit.
- Use **POST** so that credentials travel in the body and never appear in the URL.
- Store passwords on the server only as salted hashes (bcrypt, scrypt or Argon2), never as plain text.
- Add a **CSRF token** to the form and set session cookies with *Secure*, *HttpOnly* and *SameSite*.
- Validate and sanitize all input on the server to prevent SQL injection and XSS.

- Limit login attempts (rate limiting, account lockout or CAPTCHA) and return generic error messages such as "Invalid username or password".
- Offer multi-factor authentication for stronger protection.